

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

16/01/2023

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: ☐ Zanero ☐ Carminati ☐ Barenghi

Have you done any challenges/homework, even partially? ☐ Yes ☐ No

Do you want to withdraw? ☐ Yes ☐ No

Priority Grading ☐ Yes, Motivation:
☐ No

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 16 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book." The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**

- On the computer, you can open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct exam delivery.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
 - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

Consider object tracking devices such as the one developed by Apple or Tile. They are hardware devices designed to act as object finders, which help people find personal objects (e.g., keys, bags, apparel, small electronic devices, and vehicles).

Consider the following infrastructure implementation:

A user can use an app installed on smartphones to help locate the item if it is lost. In particular, to locate lost items, they use a crowdsourced network of devices: approximately billions of devices worldwide that detect and anonymously report emitted Bluetooth Low Energy 4.0 radio technology signals. If the tracking device is reported as lost and comes within range of any smartphone running the tracking app, the nearby user's app will send the item's owner an anonymous update of the lost item's location.

Each tracking device comes with a built-in speaker, which can be triggered by the user through the app to play a sound to aid in locating items at close range.

The tracking devices usually also have a "Find My Phone" feature, which can be used to produce a sound on the user's paired smartphone when the user presses a button on the device. Tracking devices usually have built-in batteries with a battery life of about one year that can be easily replaced.

[1 point] 1. What are the main assets at risk in such a scenario? Suggest at least two assets.

Asset	Motivation

1. **Personal information and location data:** As users rely on these tracking devices to locate lost items, they may also be sharing personal information and location data with the tracking device's infrastructure and operating companies, as well as other users in the crowdsourced network. This data could be at risk if a security breach occurs or if the data is mishandled by the company.
2. **Physical assets:** The physical assets that the tracking devices are being used to locate, such as keys, bags, apparel, small electronic devices, and vehicles, are also at risk if lost or stolen. The tracking devices may help users locate these assets, but they do not provide any protection against their loss or theft.
3. **Network and infrastructure:** The tracking devices rely on a network of smartphones and a centralized infrastructure to locate lost items, which could be at risk if it is compromised by a cyber attack or other security breaches.
4. **Business reputation:** If the tracking device infrastructure and operating companies fail to protect users' personal information and location data, or if there are issues with the device's performance, it could lead to negative publicity and damage the company's reputation.

[1 point] 2. Suggest, in rough order of prevalence (i.e., frequency), the most likely potential security threats against such infrastructure and their operating companies.

Threats
1) Motivation:
2) Motivation:

1. Privacy concerns: As the tracking devices use a crowdsourced network to locate lost items, there could be concerns about users' privacy as their location data is being shared anonymously with other users.
2. Security breaches: These tracking devices rely on a network of smartphones and a centralized infrastructure, a security breach in the system could potentially expose the personal information and location data of users.
3. Physical tampering and theft: The tracking devices themselves could be tampered with or stolen, potentially exposing personal information and location data, or rendering the device useless.
4. Malware and cyber attacks: The tracking devices infrastructure and operating companies may be targeted by malware or cyber attacks, which could compromise personal information and location data, as well as disrupt the service.
5. Denial of service: The infrastructure and operating companies may be targeted by a denial of service attack, which could disrupt the service and prevent users from being able to locate lost items.

[1 point] 3. Suggest, in rough order of prevalence (i.e., frequency), the most likely potential security threat agents against such infrastructure and their operating companies.

Threats
1) Motivation:
2)

Motivation:

1. Hackers and cybercriminals: These may include individuals or groups that use various techniques to gain unauthorized access to the tracking device's network and infrastructure and steal or misuse the personal information and location data of users.
2. Insider threats: These may include current or former employees of the tracking device companies who have access to sensitive information and could misuse it for personal gain or to disrupt the service.
3. State-sponsored actors: These may include nation-states or agents targeting the tracking device companies for political or strategic reasons.
4. Competitors: These may include other companies that operate in the same space and may try to gain an advantage by stealing proprietary technology or information.

[1 point] 4. Suggest, in rough order of prevalence (i.e., frequency), the most likely potential security solutions that can be deployed to counter the threats and threat agents identified in the previous questions.

1. Encryption: Encrypting personal information and location data, both in transit and at rest, can help protect it from unauthorized access and misuse.
2. Multi-factor authentication: Implementing multi-factor authentication, such as using a password and a biometric factor, can ensure that only authorized users can access personal information and location data.

Exercise 2 - Introduction to Cryptography (6 points)

[3 points] Consider the case of an air-gapped embedded device (i.e., without any network connection or similar communication means), where it is desired to have firmware updates performed connecting a mass storage device to it. The device does not have any permanent energy source (battery or connection to mains), and is thus turned off between uses. Argue on whether it is possible or not to employ digital certificates and a PKI infrastructure to validate the origin authenticity of the firmware updates. In case this is possible, sketch briefly the firmware validation procedure, otherwise highlight the reason why it cannot be performed, and provide a solution.

An air-gapped device without permanent power supply is not able to keep a reference to the passing of time, as a consequence, it is not possible to perform certificate validation due to the inability of testing for certificate expiration.

This shortcoming can either be circumvented switching to an alternate firmware validation approach (e.g., employing a static public key to perform signature verification), or obtaining a time reference. A viable approach to limit the connection of the device with the outer world is to decode the MSF-60 radio time reference, or to employ a GPS module. Note that, while at considerable effort, both can be spoofed.

[3 points] A PIN selection form asks the user to pick a 6 decimal digits number to be employed to unlock a smartphone. Considering the case where a certain amount of users pick their birthdate, encoded as YYMMDD (least significant digits of the year, followed by month and day of birth) as their PIN, what is the average amount of attempts you have to make to guess one of their PINs?

Is it larger, smaller or about the same amount of attempts you have to make to guess a random sweep pattern over all the dots in a 2 x 4 grid?

A randomly chosen birthdate expressed as above is selected among 365×100 possible ones (assuming that no clue about the age of the user is available). Ignoring the bias in the dates of birth (which are not uniformly distributed over the year), this implies that about 36500 guesses will yield a correct PIN, and, on average $36500/2$ guesses will be sufficient.

A sweep pattern over a 4 x 2 grid is represented by the order in which the eight dots are connected in the sweep: this yields $8!$, that is 40320, possible sweep patterns, making a sweep pattern over a 4 x 2 grid only a little harder to guess than a random birthdate.

Exercise 3 - Binary Vulnerabilities (6 points)

```
01 #include <stdio.h>
02
03 typedef struct
04 {
05     char buf2[15];
06     char buf[64];
07 } data_t;
08 // This function writes 5 times 15 char in data.buf
09 int scramble(int *sequence)
10 {
11     data_t data;
12
13     for (int i = 0; i < 5; i++)
14     {
15         scanf("%15s", data.buf2);
16         strncpy(&data.buf[sequence[i]*15], data.buf2, 15);
17         printf(data.buf);
18     }
19 }
20
21 void main()
22 {
23     int sequence[5] = {3,1,4,0,2};
24
25     scramble(sequence);
26 }
```

- The C standard library is loaded at a known address during every execution of the program, and that the address of the function `system()` is `0xf4d0e2d3`.
- Environment variables are located in the highest addresses.
- *The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdec1** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against exploiting memory corruption errors are present (unless specified otherwise).*

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	Line	Motivation
Buffer Overflow	16	When “i” is 4, the <code>strncpy</code> copies from offset 60 to offset 75 of the array “sequence”

Format String	17	No format specified in the printf
---------------	----	-----------------------------------

[2 points] 2. **Focus only on the stack-based buffer overflow(s) you found.** Write an exploit for this vulnerability that **must execute the following shellcode**, composed of 8 bytes, which opens a shell: **0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90**.

2.a) Describe **all the steps and assumptions** required for the successful exploitation of the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, (if any), the input provided during the execution, if multiple executions are necessary.

The shellcode is stored in the environment variables.

During the third iteration, the following input will overwrite the savedEIP:

"AAAABBBB<addr_shellcode>"

Where the As fill the end of the buffer, the Bs overwrite the savedEBP, and the add_shellcode (4 bytes) overwrite the savedEIP

2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

High	buf2[0-3]	<- ESP	scamble()	AAAA	<- ESP
^	buf2[4-7]		scamble()	BBBB	
	buf2[8-11]		scamble()	&shellcode	
	buf2[12-14]		scamble()	(not relevant)	
	buf[0-3]		scamble()	(not relevant)	
	...		scamble()	...	
	buf[60-63]		scamble()	AAAA	
	savedEBP	<- EBP	scamble()	BBBB	<- EBP
	savedEIP		main()	&shellcode	
	&sequence		main()	&sequence	
low	savedEBP		main()	savedEBP	

[2 points] 3. **Focus only on the format string(s) you found.**

Write an exploit for this vulnerability that executes the previous shellcode, assuming that you have already prepared the memory (the shellcode has been positioned in a place under the control of the attacker) with the correct arguments. Assume that:

- The address of the return address (*saved EIP*) of the function exploited is: **0x8da0fee4** (i.e., where to write)
- The address of the first instruction of the **shellcode** is at **0xd3f4e2d0** (i.e., what to write)
- The displacement on the stack of the vulnerable function's argument is: **7**

Knowing that $dec(0xd3f4) = 54260$ and $dec(0xe2d0) = 58064$, write the exploit clearly, describe **all the steps, assumptions and the components of the format string** required to successfully exploit the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution.

The format string exploit is the following:

“\xe6\xfe\xa0\x8d\xe4\xfe\xa0\x8d%(54260-8)c%7\$hn%(58964-54260)c%8\$hn”

I need to split this exploit in 2 parts:

The first part must be provided during the second iteration, while the second part during the last one.

[1 point] Assume now that the main function is modified as follows:

```
21 void main()
22 {
23     int sequence[5];
24     srand(0); // seed the random number generator with seed=0
25
26     for (int i = 0; i < 5; i++) {
27         sequence[i] = rand() % 5; // generate a random number between 0 and 4
28     }
29
30     scramble(sequence);
31 }
```

Is any of the previous exploits working without any modification? If yes explain why, if not motivate your answer and provide an alternative solution: describe **all the steps and assumptions** required for the successful exploitation of the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, (if any), the input provided during the execution, if multiple executions are necessary.

Both exploits work if the sequence generated is the same. In particular, for the FS exploit, it's necessary to have index 1 in the second position and index 0 in the last position. For the Buffer overflow, it's necessary to have index 4 in the third position.

It is important to note that the random sequence is always the same at every run of the program since the seed is always the same(0). Hence, it is possible to know the random sequence.

To make the Buffer overflow work, we need one index of the sequence to be 4. Otherwise, we will never overflow.

To make the Format string exploit work, we need two indexes of the sequence to be 0 and 1. Otherwise, we will never compose the format string exploit.

Exercise 4 - Web Vulnerabilities (6 points)

OpenMessages.com is an open-source web and mobile application for messaging. Users can freely send messages to their contacts, relying on end-to-end encryption to protect their messages. Moreover, by being open-source, anybody can check the source code and contribute to improving the application.

The relevant code of the application is the following:

```
00 app.route("/send_message") # /send_message
01 def send_message(request, session):
02     if not is_session_valid(session):
03         return redirect(url_for("/login"))
04     if request.method != "GET":
05         return abort(403)
06     my_number = session["my_number"]
07     contact_number = request.args["contact_number"]
08     message = request.args["message"]
09     query = "SELECT * FROM USERS WHERE number = '%s';"
10     result = db_execute_query_with_statement(query, contact_number)
11     if len(result) == 0:
12         return render_template("send_message.html", error="Contact not found!")
13     query = "INSERT INTO MESSAGES VALUES ('%s', '%s', '%s');"
14     db_execute_query_with_statement(query, message, my_number, contact_number)
15     return redirect(url_for("/home"))
16
17 app.route("/private_messages") # /private_messages
18 def private_messages(request, session):
19     if not is_session_valid(session):
20         return redirect(url_for("/login"))
21     if request.method != "GET":
22         return abort(403)
23     my_number = session["my_number"]
24     contact_number = request.args["contact_number"]
25     query = "SELECT sender, message FROM MESSAGES WHERE (receiver = '" + my_number + "' AND
26             sender = '" + contact_number + "') OR (receiver = '" + contact_number + "' AND sender
27             = '" + my_number + "');"
28     messages = db_execute_query(query)
29     return render_template("private_messages.html", messages=messages)
30
31 # private_messages.html
32 <html>
33 ...
34 {% for message in messages %} # Iterating over messages
35     <div>
36         <h7>From: {{ message["sender"] }}</h7>
```

```

35         <p>{{ message["message"] }}</p>
36     </div>
37     {% endfor %}
38     ...
39 </html>

```

The relational database schema used by the website is the following:

Table: **USERS**

number (Primary Key) (String)	username (String)	email (String)	password_hash (Bytes)
----------------------------------	----------------------	-------------------	--------------------------

Table: **MESSAGES**

message (String)	sender (String) (Primary Key) (Referencing USERS.number)	receiver (String) (Primary Key) (Referencing USERS.number)
---------------------	--	--

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **db_execute_query_with_statement** correctly executes queries with prepared statements.
- The function **render_template** **does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT ... ; INSERT INTO ... ;" fails if executed)
- The value of session["my_number"] is the correct mobile phone number of the logged user.

More details:

- The function **render_template** generates an html page with optional parameters that are placed in the html code.
- The function **url_for** generates an url of the provided page of the same domain. Optional query parameters of the url can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided url.
- The function **db_execute_query_with_statement** returns a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, result[4]["comment"] is accessing the field "comment" of the 5th row.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

<i>Vulnerability class</i>	<i>Is there a vulnerability of this class in the code above? How could an adversary exploit it?</i>	<i>If the vulnerability is present, explain the simplest procedure to remove it.</i>
SQL Injection (SQL-I)	<i>Line 25. There's a SQL-Injection vulnerability. The attacker may leak data from the DB.</i>	<i>Use prepared statements.</i>
cross-site scripting (XSS) Specify if reflected, stored, DOM...	<i>The input message in send message can result in a stored XSS at line 35. The attacker may execute arbitrary javascript in the browser of the victim.</i>	<i>The simplest procedure would be to apply escaping/filtering to the message.</i>
Cross-site request forgery (CSRF)	<i>Yes, there is no implementation of a CSRF token.</i>	<i>See slides (CSRF tokens, etc.).</i>

[1.5 points] 2. One of your friends sent you a message on OpenMessages.com. After visiting the page of private messages, you notice that you sent a message to the number +39 333 4455666 with the content: "This application is terrible, better stick with WhatsApp". Can you reproduce the exploit? Explain all the steps you'd take to reproduce the exploit, including the final code you'd write.

To reproduce the exploit you just need to send a message to the victim with the content:

```
<script>document.location="https://OpenMessage.com/send_message?contact_number=+393334455666&message=This application is terrible, better stick with WhatsApp"</script>
```

[1.5 points] 3. Without sending any message, can you leak the conversation between +39 333 1122333 and +39 333 7788999? Explain all the steps you'd take to reproduce the exploit, including the final code you'd write.

Exploiting the SQL-Injection:

contact_number = '') OR (receiver = '+39 333 1122333' AND sender '+39 333 7788999') OR (receiver = '+39 333 7788999' AND sender '+39 333 1122333') ; -- "

The comment at the end is needed, otherwise the query will fail.

Exercise 5 - Network Security (6 points)

Consider the DNS protocol, and specifically consider how the process of requesting a recursive DNS lookup works (assume this simplification to hold true): When a client needs to perform a DNS lookup, it generates a (usually UDP) DNS query packet (of an average length of 50 Bytes). The local DNS server receives the packet, and checks whether it has stored in cache one or more matches for the given domain name. If so, it responds with a DNS lookup response packet (of an average length of 250 Bytes), that provides the IPv4 address related to the requested domain name. If, however, the local DNS server does not have a cached match for the domain name, it contacts the authoritative name server related to that domain name, which provides the local DNS server with the requested information. Note that a request to the authoritative DNS server can ask either for a specific record, e.g., the IPv4 address for the given domain name, or for all the available records at the same time, e.g., IPv4 address, IPv6 address, mail server address, authoritative name servers for the domain... (assume that each record adds an average length of 250 Bytes to the response). Finally, the local DNS server forwards only the necessary information to the client, as it would have done if it already cached the files.

1. [2 points] The DNS protocol, specifically the part that has been described above, can be “misused” to ease a specific network attack. Please explain which attack, making sure to mention the “feature” of the protocol that allows it to be misused, and briefly describe its working.

DoS attack based on amplification factors of the Authoritative DNS responses. The attacker can spoof the IP address of the victim and send to the authoritative DNS server a request for all the available records. In this way, the Authoritative DNS response, way bigger than the query, is sent to the victim.

2. [1 points] We discovered in the wild an attack that exploits this protocol. The network administrators of the company that was hit by this attack were able to capture the following headers of some suspect packets at their network’s border firewall:

Src	Dest	Type
IP 86.86.230.18 (src port 53)	> 105.5.1.230 (dst port 53)	UDP DNS Resp., length 2433
IP 86.86.230.18 (src port 53)	> 105.5.1.230 (dst port 53)	UDP DNS Resp., length 2433

IP 177.122.9.3 (src port 53)	> 105.5.1.230 (dst port 53)	UDP DNS Resp., length 2612
IP 177.122.9.3 (src port 53)	> 105.5.1.230 (dst port 53)	UDP DNS Resp., length 1940
IP 86.86.230.18 (src port 53)	> 105.5.1.230 (dst port 53)	UDP DNS Resp., length 2433
IP 177.122.9.3 (src port 53)	> 105.5.1.230 (dst port 53)	UDP DNS Resp., length 2612
IP 177.122.9.3 (src port 53)	> 105.5.1.230 (dst port 53)	UDP DNS Resp., length 2612
IP 177.122.9.3 (src port 53)	> 105.5.1.230 (dst port 53)	UDP DNS Resp., length 1940

Can you tell the attacker's IP? And the victim? What type of server would you assume is sending the packets?

Not the attacker IP, since it's the DNS server which "answers" the victim, but the victim is 105.5.1.230. The type of server is definitely an authoritative DNS Server, since the length of the answers is of around 2000 Bytes, which means that the answers have around 8 records

3. [1 point] As the network security administrator for the network hit by this attack (i.e., you control the border firewall and can add arbitrary rules), can you mitigate the effect of this attack or prevent it altogether? Why?

Partially Yes: The network admin can set up a stateful firewall to ensure that DNS responses are not forwarded if no DNS query from a client has been provided, or generally filter the rate of DNS responses that it allows towards a specific client. However, this solution only moves the attack to the Firewall, which can be DoSsed too with enough power.

4. [1 point] You are the network security administrator for the DNS servers exploited to execute this attack (i.e., you have control on the responses to the queries you receive), can you mitigate the effect of this attack or prevent it altogether? Why?

Yes, you can avoid to answer to requests for all records, removing the amplification factor

5. [1 point] Assume that now DNS is implemented over TCP instead of UDP. Would this change mitigate the effect of this attack? Why?

Yes, since the attacker would not be able to exploit the amplification factor against the victim, since the DNS server would SYN-ACK the victim and not complete the three way handshake.

Exercise 6 - Malware (6 points)

Consider the following pseudo-code, which describes a code excerpt taken from a malicious program.

```
1. if is_debugger_present():
2.   exit(-1)
3. else:
4.   counter = 0
5.   while True:
6.     counter += 1
7.     if counter - sys.get_system_time() > 0:
8.       break
```

6.1 [3 points] Name the evasive techniques implemented by this pseudo-code. Specify the first and last instruction line for each. Please note: naming wrong techniques will make you lose points.

Anti-debugging lines 1-2
Dormant code lines 4-8

6.2 [2 points] Why is signature-based detection usually ineffective against metamorphic malware?

Metamorphic malware rewrites itself when propagating. Signature-based is essentially a blacklisting approach, which tries to detect malware by looking for some invariant parts known to belong to some malware family. Given that the number of possible versions that a metamorphic engine can generate is virtually infinite, there are in theory no “invariant” parts and therefore the signature-based approach cannot be used.

6.3 [1 point] Suppose the malicious program mentioned above is also metamorphic. How would you detect it?

Supposing the metamorphic engine leaves no invariant parts when propagating, signature based is not an option.

A naive debugging-based approach would make the program exit, so not an option either.

You have two valid options (need to specify only one for full mark):

1. (debugging-based detection) You debug the program while patching the `is_debugger_present` API so that it returns false. At this point it would be advantageous to also patch `sys.get_system_time`.

2. (heuristics-based detection) You run it in a virtual machine without any active debugger and then assess the state of the machine after execution is over. It may also be good to observe the outgoing traffic of the VM.